

Group Members: Leda Ning, Junchen Liu, Samuel Green, Kaden Chan

A.

Project Description

Jungle Library is a library for all ages. Jungle Library seeks to implement a centralized relational database system to streamline and structure its operational processes. The proposed database design will incorporate the following core entities: Person, Employee, Member, Book, Publisher, Author, and Payment, ensuring data consistency, integrity, and efficient management of library resources and transactions.

A Person entity may represent either an Employee or a Member, with the possibility that Employee can also be Member. The system shall store the following attributes for each Person: Person ID, Name (First Name, Middle Name, Last Name), Address, Gender, Date of Birth, and Phone Number(s), where a Person may have multiple associated phone numbers. An Employee must be at least 18 years of age. The Person ID attribute must follow the format "PXXX", where X denotes a digit between 0 and 9.

Each Member is issued a library card, for which details such as Card ID, Issue Date, Membership Level (Silver or Gold), and other relevant information are maintained. The library may also associate Promotions with library cards. Each Promotion is defined by a unique Promotion code and an accompanying description.

Each Member is categorized as either Silver or Gold. For Gold Members, a Guest Log is maintained, capturing details such as the Gold Member's Card ID, Guest ID, Guest Name, Guest Address, and Guest Contact Information. Temporary IDs are assigned to individuals visiting as guests of a Gold Member. Each Guest ID is unique only within the scope of the associated Gold Member and is not globally unique across the entire system.

An Employee may belong to one of three categories: Library Supervisor, Cataloging Manager, or Receptionist. The system records the Employment Start Date for each Employee. A Receptionist is required to receive training from a designated Trainer, where a Trainer may be either a Library Supervisor or a Cataloging Manager. Both Library Supervisors and Cataloging Managers are permitted to train multiple Receptionists.

Details of Books, including Book ID, Book Title, and other relevant information, are maintained in the system. Books are classified into three categories: Cate. 1, Cate. 2, and Cate. 3. Each Cataloging Manager is responsible for cataloging one category per day, though they may work on different categories on different days. Additionally, a Person may leave Comments on Books, which include the Comment Time, a Rating Score (ranging from 1 to 5), and the Comment Content.

A Publisher may publish multiple Books, whereas each Book is assumed to be published by a single Publisher. The system stores Publisher details, including Publisher ID, Publisher Name,

and other relevant information (additional attributes may be defined based on requirements). Similarly, Author details, such as Author ID, Author Name, and other pertinent information, are maintained. A Book may have multiple Authors, and an Author may contribute to multiple Books.

A Receptionist is responsible for maintaining records of Borrowing details. Each record includes information about the borrowed Book, Issue Date, Due Date for Return, the Person borrowing the book, the responsible Receptionist, and associated Payment Details. Borrowing records are created only when a Person borrows a Book. Payment Details include Payment ID, Payment Method (e.g., Cash, Debit/Credit Card), Payment Time, and Amount Paid.

In addition, the Receptionist is responsible for managing Member Inquiries. Each inquiry is assigned a unique Inquiry ID and records details such as the Inquiry Time and Resolution Status. To monitor member satisfaction and engagement, Members may provide a Rating (ranging from 1 to 5) for the inquiries they have submitted.

B.

Project Questions:

Q1. Is the ability to model superclass/subclass relationships likely to be important in the library management system like above? Why or why not?

- Yes. The ability to model superclass/subclass relations is important for the library management system because of the subcategories of employees like library supervisor, cataloging manager, and receptionist. Additionally, Member has subcategories for Silver and Gold. These subcategories are ideal for modeling with super/sub-class relationships.

Q2. Can you think of 5 more rules (other than those explicitly described above) that are likely to be used in the above environment? Please describe how your design would be changed to satisfy your additional rules?

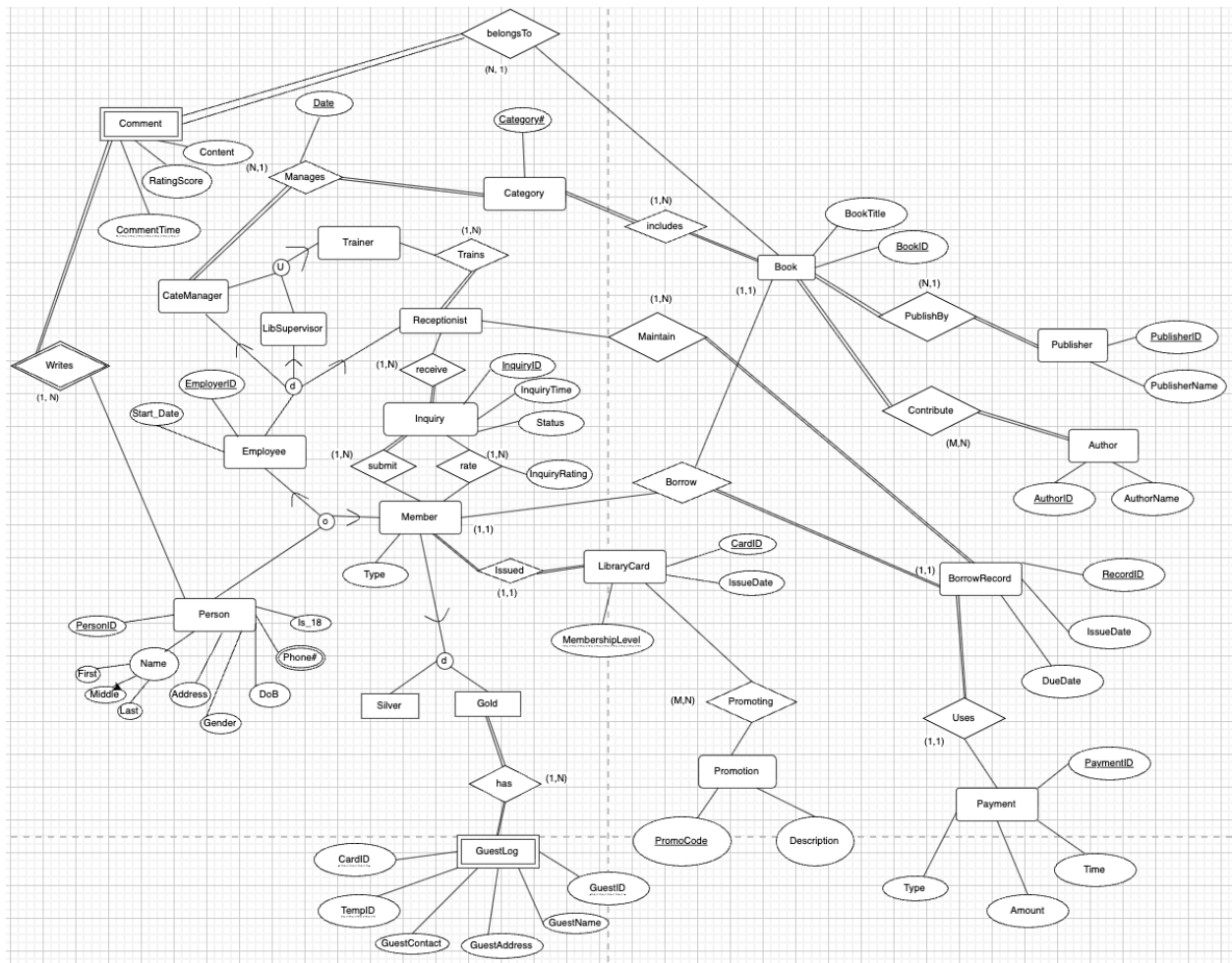
1. A guest can only check-out/borrow 1 book at a time.
 - To enforce this, add a new 'checkedOut' attribute to GuestLogb and a new relation called 'GuestBorrow' with a FK to the guestID to keep track of book borrows specifically from guests. Whenever a guest checks out a book via the 'GuestBorrow' relation, a new BorrowRecord is created and the 'checkedOut' attribute is set to 1 which signifies a book is checked out by the guest. A trigger will enforce the constraint that if the 'checkedOut' attribute of the guest is 1, then it won't allow them to borrow a new book.
2. Every time a member has an overdue book, they get a demerit
 - To enforce this, we add a demeritCount attribute to the Member relation. We then define a trigger on the book return operation (e.g., update of BorrowRecord when ReturnDate is set). If ReturnDate > DueDate, the trigger automatically increments demeritCount for that member by 1. This keeps the demerit total consistent with overdue behavior.
3. After 5 demerits, a member will lose its membership (membership will be canceled)

- To enforce this, add the above rules and then add an additional status attribute to the Member relation that is set to 'active' whenever a membership is created and then set to 'canceled' when 'demeritCount' reaches 5 via a trigger.
4. There is a one-time use Buy-one, Get-one free (BOGO) promotion code for the first month of every membership
- To enforce this, we use the existing Promotion and Promoting tables. A trigger on Promoting checks two things when a BOGO promo code is inserted: (1) the current date is within one month of the card's IssueDate, and (2) the same promo code has not already been used for that card. Otherwise, the insert is rejected.
5. Only Library Supervisors are allowed to add new books into the system.
- We use the existing LibSupervisor and Book tables. We add an attribute SupervisorID to Book with a foreign key to LibSupervisor(EmployeeID). New books must be inserted with a valid SupervisorID. This ensures that every book in Book is recorded as having been added by a Library Supervisor, and any attempt to insert a book without a valid supervisor ID will fail the foreign key constraint.

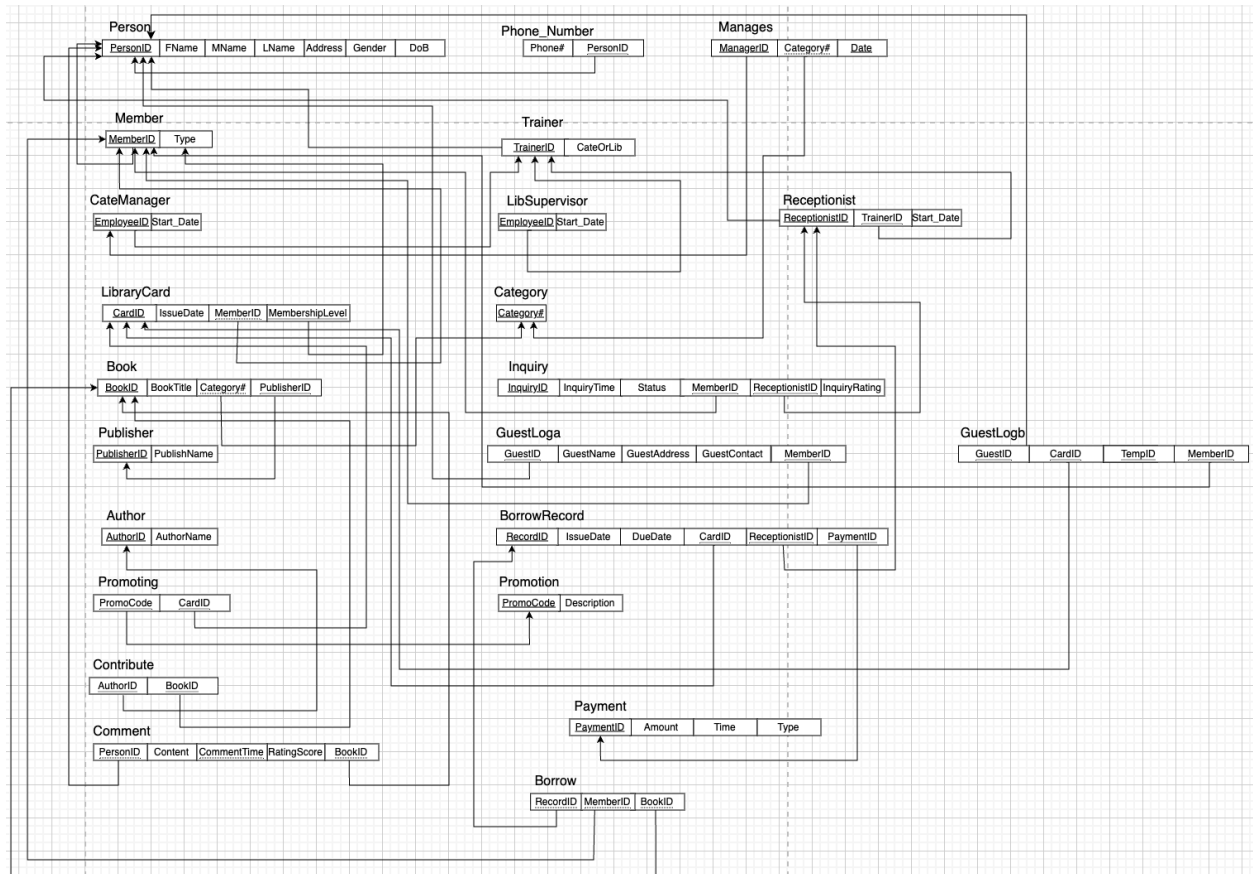
Q3. Justify the use of a Relational DBMS like Oracle for this project (Successfully design a relational database system and show all implementation in the final report at Phase IV).

- A relational DBMS like Oracle is very beneficial for a project like this because relational DBMS's provide a very flexible and efficient means of storing and changing data for the various requirements in the project. The library management system involves a lot of transactions on data like creating borrow records and storing book info and comments as well as storing employee and member information. A DBMS provides tools to precisely represent all these relations, automatically enforce constraints, control redundancy, and provide security and recovery services.

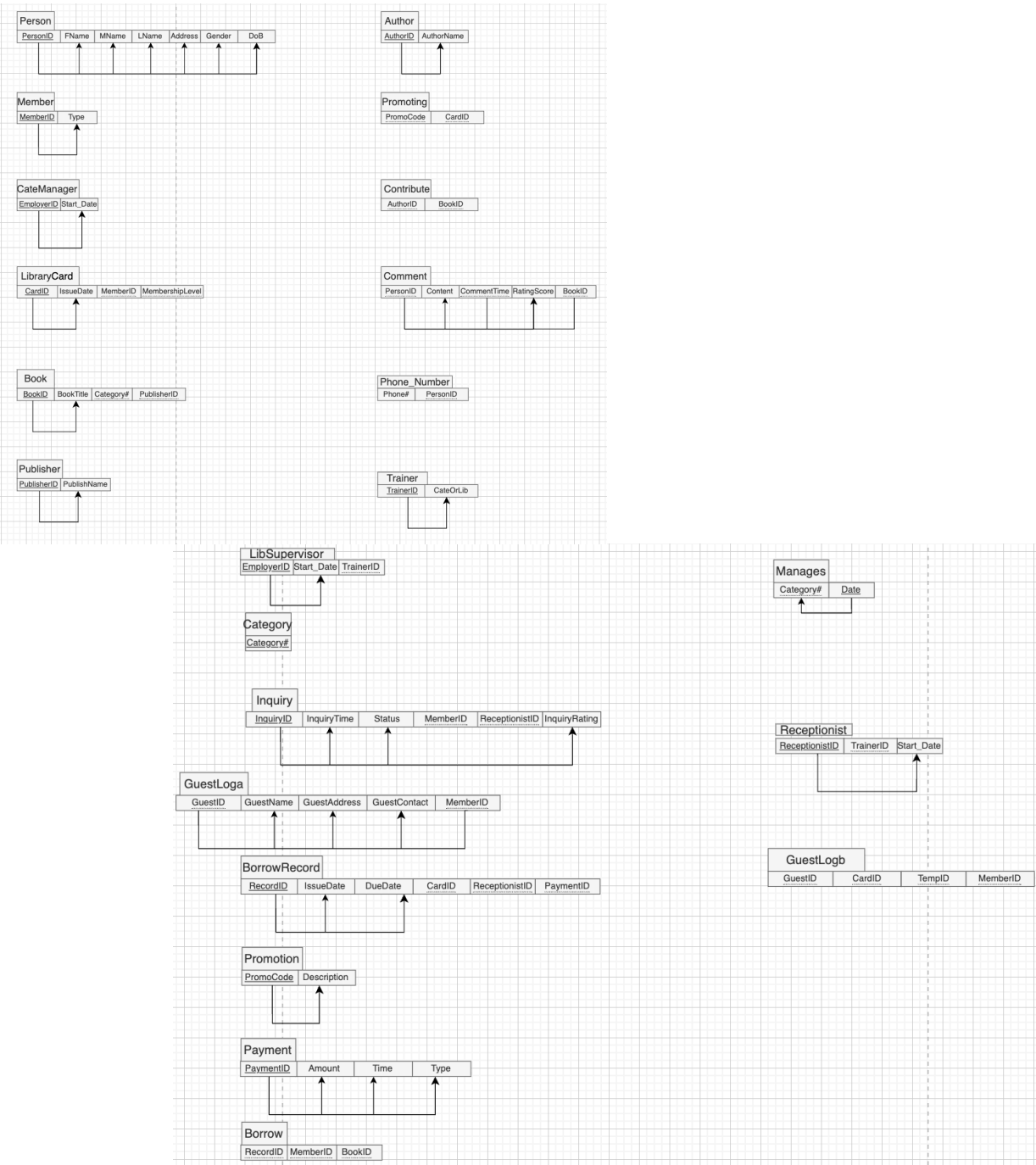
Part C:



Part D:



Part E:



Part F:

Create table for each relation

CREATE TABLE Person (

```
    PersonID CHAR(4) NOT NULL,  
    FName VARCHAR(100) NOT NULL,  
    MName VARCHAR(100),  
    LName VARCHAR(100) NOT NULL,  
    Address VARCHAR(255) NOT NULL,  
    Gender VARCHAR(20) NOT NULL,  
    DoB DATE NOT NULL,  
    PRIMARY KEY (PersonID),  
    CHECK (REGEXP_LIKE(PersonID, '^P[0-9]{3}$'))  
);
```

CREATE TABLE PhoneNumber (

```
    PersonID CHAR(4) NOT NULL,  
    PhoneNo CHAR(10) NOT NULL,  
    PRIMARY KEY (PersonID, PhoneNo),  
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID)  
);
```

CREATE TABLE Member (

```
    MemberID CHAR(4) NOT NULL,  
    Type VARCHAR(6) NOT NULL,  
    PRIMARY KEY (MemberID),  
    FOREIGN KEY (MemberID) REFERENCES Person(PersonID),  
    CHECK (Type IN ('Silver', 'Gold'))  
);
```

CREATE TABLE Trainer (

```
    TrainerID CHAR(4) NOT NULL,  
    CateOrLib VARCHAR(20) NOT NULL,  
    DoB DATE NOT NULL,  
    PRIMARY KEY (TrainerID),  
    FOREIGN KEY (TrainerID) REFERENCES Person(PersonID),  
    FOREIGN KEY (DoB) REFERENCES Person(DoB),  
    CHECK (CateOrLib IN ('LibSupervisor', 'CateManager'))  
);
```

CREATE TABLE CateManager (

```
    EmployeeID CHAR(4) NOT NULL,  
    Start_Date DATE NOT NULL,  
    PRIMARY KEY (EmployeeID),
```

```
FOREIGN KEY (EmployeeID) REFERENCES Trainer(TrainerID)
);
```

```
CREATE TABLE LibSupervisor(
    EmployeeID CHAR(4) NOT NULL,
    Start_Date DATE NOT NULL,
    PRIMARY KEY (EmployeeID),
    FOREIGN KEY (EmployeeID) REFERENCES Trainer(TrainerID)
);
```

```
CREATE TABLE Receptionist (
    ReceptionistID CHAR(4) NOT NULL,
    TrainerID CHAR(4) NOT NULL,
    Start_Date DATE NOT NULL,
    PRIMARY KEY (ReceptionistID),
    FOREIGN KEY (ReceptionistID) REFERENCES Person(PersonID),
    FOREIGN KEY (TrainerID) REFERENCES Trainer(TrainerID)
);
```

```
CREATE TABLE LibraryCard (
    CardID VARCHAR(10) NOT NULL,
    IssueDate DATE NOT NULL,
    MemberID CHAR(4) NOT NULL,
    MembershipLevel VARCHAR(6) NOT NULL,
    PRIMARY KEY (CardID),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    CHECK (MembershipLevel IN ('Silver', 'Gold'))
);
```

```
CREATE TABLE Promotion (
    PromoCode CHAR(9) NOT NULL,
    Description VARCHAR(500),
    PRIMARY KEY (PromoCode)
);
```

```
CREATE TABLE Promoting (
    PromoCode CHAR(9) NOT NULL,
    CardID VARCHAR(10) NOT NULL,
    PRIMARY KEY (PromoCode, CardID),
    FOREIGN KEY (PromoCode) REFERENCES Promotion(PromoCode),
    FOREIGN KEY (CardID) REFERENCES LibraryCard(CardID)
);
```

```
CREATE TABLE Category (
```



```
CategoryNo VARCHAR(8) NOT NULL,  
PRIMARY KEY (CategoryNo),  
CHECK (CategoryNo IN ('Cate. 1', 'Cate. 2', 'Cate. 3'))  
);
```

```
CREATE TABLE Manages(
```

```
ManagerID CHAR(4) NOT NULL,  
CategoryNo VARCHAR(8) NOT NULL,  
Date DATE NOT NULL,  
PRIMARY KEY (ManagerID, CategoryNo, Date),  
FOREIGN KEY (ManagerID) REFERENCES CateManager(EmployeeID),  
FOREIGN KEY (CategoryNo) REFERENCES Category(CategoryNo)  
);
```

```
CREATE TABLE Publisher (
```

```
PublisherID CHAR(9) NOT NULL,  
PublishName VARCHAR(50),  
PRIMARY KEY (PublisherID)  
);
```

```
CREATE TABLE Book (
```

```
BookID VARCHAR(20) NOT NULL,  
BookTitle VARCHAR(200) NOT NULL,  
CategoryNo VARCHAR(8),  
PublisherID CHAR(9) NOT NULL,  
PRIMARY KEY (BookID),  
FOREIGN KEY (CategoryNo) REFERENCES Category(CategoryNo),  
FOREIGN KEY (PublisherID) REFERENCES Publisher(PublisherID)  
);
```

```
CREATE TABLE Author (
```

```
AuthorID CHAR(9) NOT NULL,  
AuthorName VARCHAR(50),  
PRIMARY KEY (AuthorID)  
);
```

```
CREATE TABLE Contribute (
```

```
AuthorID CHAR(9) NOT NULL,  
BookID VARCHAR(20) NOT NULL,  
PRIMARY KEY (AuthorID, BookID),  
FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID),  
FOREIGN KEY (BookID) REFERENCES Book(BookID)  
);
```

```
CREATE TABLE Comment (  
    PersonID CHAR(4) NOT NULL,  
    Content VARCHAR(2100),  
    CommentTime TIMESTAMP NOT NULL,  
    RatingScore INT,  
    BookID VARCHAR(20) NOT NULL,  
    PRIMARY KEY (PersonID, CommentTime, BookID),  
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID),  
    FOREIGN KEY (BookID) REFERENCES Book(BookID),  
    CHECK (RatingScore BETWEEN 1 AND 5)  
);
```

```
CREATE TABLE Inquiry (  
    InquiryID CHAR(10) NOT NULL,  
    InquiryTime TIMESTAMP NOT NULL,  
    Status BOOLEAN,  
    MemberID CHAR(4) NOT NULL,  
    ReceptionistID CHAR(4) NOT NULL,  
    InquiryRating INT,  
    PRIMARY KEY (InquiryID),  
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),  
    FOREIGN KEY (ReceptionistID) REFERENCES Receptionist(ReceptionistID),  
    CHECK (InquiryRating BETWEEN 1 AND 5)  
);
```

```
CREATE TABLE GuestLoga (  
    GuestID CHAR(4) NOT NULL,  
    GuestName VARCHAR(50),  
    GuestAddress VARCHAR(255),  
    GuestContact VARCHAR(50),  
    MemberID CHAR(4) NOT NULL,  
    PRIMARY KEY (GuestID, MemberID),  
    FOREIGN KEY (GuestID) REFERENCES Person(PersonID),  
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID)  
);
```

```
CREATE TABLE GuestLogb (  
    GuestID CHAR(4) NOT NULL,  
    CardID VARCHAR(10) NOT NULL,  
    TempID VARCHAR(9) NOT NULL,  
    MemberID CHAR(4) NOT NULL,  
    PRIMARY KEY (GuestID, CardID, TempID, MemberID),  
    FOREIGN KEY (CardID) REFERENCES LibraryCard(CardID),  
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
```

);

CREATE TABLE Payment (

PaymentID VARCHAR(10) NOT NULL,
Amount NUMBER(10, 2) NOT NULL,
Time TIMESTAMP,
Type VARCHAR(20),
PRIMARY KEY (PaymentID)
);

CREATE TABLE BorrowRecord (

RecordID VARCHAR(10) NOT NULL,
IssueDate DATE NOT NULL,
DueDate DATE NOT NULL,
CardID VARCHAR(10) NOT NULL,
ReceptionistID CHAR(4) NOT NULL,
PaymentID INT NOT NULL,
PRIMARY KEY (RecordID),
FOREIGN KEY (CardID) REFERENCES LibraryCard(CardID),
FOREIGN KEY (ReceptionistID) REFERENCES Receptionist(ReceptionistID),
FOREIGN KEY (PaymentID) REFERENCES Payment(PaymentID)
);

CREATE TABLE Borrow (

RecordID VARCHAR(10) NOT NULL,
MemberID CHAR(4) NOT NULL,
BookID VARCHAR(20) NOT NULL,
PRIMARY KEY (RecordID, MemberID, BookID),
FOREIGN KEY (RecordID) REFERENCES BorrowRecord(RecordID),
FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
FOREIGN KEY (BookID) REFERENCES Book(BookID)
);

PART FD

1. Create View TopGoldMember(FName, LName, IssueDate) AS SELECT FName, LName, IssueDate FROM LibraryCard L, Person P, BorrowRecord B WHERE P.PersonID = L.MemberID AND L.CardID = B.CardID AND B.IssueDate > DATE_SUB(CURDATE(), Interval 1 Month) GROUP BY B.CardID, P.FName, P.LName, L.IssueDate HAVING Count(*) > 5 ORDER BY COUNT(*) DESC LIMIT 9999999;

Oracle: CREATE OR REPLACE VIEW TopGoldMember (FName, LName, IssueDate) AS
SELECT P.FName, P.LName, L.IssueDate
FROM LibraryCard L
JOIN Person P ON P.PersonID = L.MemberID
JOIN BorrowRecord B ON L.CardID = B.CardID
WHERE B.IssueDate > ADD_MONTHS(SYSDATE, -1) -- Last 1 month
GROUP BY P.FName, P.LName, L.IssueDate
HAVING COUNT(*) > 5
ORDER BY COUNT(*) DESC;

2. Create View PopularBooks(BookID, BookTitle, Category, PublishName, AuthorNames, BorrowCount) AS SELECT B.BookID, B.BookTitle, B.Category, P.PublishName, GROUP_CONCAT(DISTINCT A.AuthorName SEPARATOR ', ') AS AuthorNames, Count(DISTINCT BR.RecordID) AS BorrowCount FROM Book B, BorrowRecord BR, Borrow BW, Publisher P, Author A, Contributor C WHERE B.PublisherID = P.PublisherID AND B.BookID = BW.BookID AND BW.RecordID = BR.RecordID AND B.BookID = C.BookID AND A.AuthorID = C.AuthorID AND BR.IssueDate > DATE_SUB(CURDATE(), Interval 1 Year) GROUP BY B.BookID, B.BookTitle, B.Category, P.PublishName ORDER BY COUNT(DISTINCT BR.RecordID) DESC LIMIT 99999999;

oracle:CREATE OR REPLACE VIEW PopularBooks (
BookID,
BookTitle,
Category,
PublishName,
AuthorNames,
BorrowCount
) AS
SELECT
B.BookID,
B.BookTitle,
B.CategoryNO,
P.PublishName,
LISTAGG(DISTINCT A.AuthorName, ', ') WITHIN GROUP (ORDER BY A.AuthorName) AS
AuthorNames,
COUNT(DISTINCT BR.RecordID) AS BorrowCount

```

FROM Book B
JOIN Publisher P ON B.PublisherID = P.PublisherID
JOIN Contributor C ON B.BookID = C.BookID
JOIN Author A ON C.AuthorID = A.AuthorID
JOIN Borrow BW ON B.BookID = BW.BookID
JOIN BorrowRecord BR ON BW.RecordID = BR.RecordID
WHERE BR.IssueDate > ADD_MONTHS(SYSDATE, -12) -- Last 1 year
GROUP BY B.BookID, B.BookTitle, B.CategoryNO, P.PublishName
ORDER BY COUNT(DISTINCT BR.RecordID) DESC;

```

3. Create View BestRatingPublisher(PublishName) AS SELECT P.PublishName FROM Publisher P WHERE NOT EXISTS (SELECT 1 FROM Book B, Comment C WHERE P.PublisherID = B.PublisherID AND B.BookID = C.BookID GROUP BY B.BookID HAVING AVG(C.RatingScore) < 4.0);

```

CREATE OR REPLACE VIEW BestRatingPublisher (PublishName) AS
SELECT P.PublishName
FROM Publisher P
WHERE NOT EXISTS (
    SELECT 1
    FROM Book B
    JOIN Comments C ON B.BookID = C.BookID
    WHERE P.PublisherID = B.PublisherID
    GROUP BY B.BookID
    HAVING AVG(C.RatingScore) < 4.0
);

```

4. Create View PotentialGoldMember(FName, LName, Phone, ID) AS SELECT P.FName, P.LName, PN.Phone, P.PersonID FROM Person P, (SELECT PersonID, MIN(Phone) AS Phone FROM Phone_Number GROUP BY PersonID) PN, Borrow B, BorrowRecord BR, Member M WHERE P.PersonID = M.MemberID AND M.Type = 'Silver' AND B.MemberID = M.MemberID AND B.RecordID = BR.RecordID AND PN.PersonID= P.PersonID AND BR.IssueDate >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR) GROUP BY P.PersonID, P.FName, P.LName, PN.Phone HAVING COUNT(DISTINCT DATE_FORMAT(BR.IssueDate, '%Y-%m')) = 12;

```

CREATE OR REPLACE VIEW PotentialGoldMember (FName, LName, Phone, ID) AS
SELECT
    P.FName,
    P.LName,
    PN.Phone,
    P.PersonID

```

```

FROM Person P
JOIN (
    SELECT PersonID, MIN(PhoneNO) AS Phone
    FROM PhoneNumber
    GROUP BY PersonID
) PN ON P.PersonID = PN.PersonID
JOIN Member M ON P.PersonID = M.MemberID
JOIN Borrow B ON M.MemberID = B.MemberID
JOIN BorrowRecord BR ON B.RecordID = BR.RecordID
WHERE M.Type = 'Silver'
    AND BR.IssueDate >= ADD_MONTHS(TRUNC(SYSDATE), -12) -- Last 1 year
GROUP BY P.PersonID, P.FName, P.LName, PN.Phone
HAVING COUNT(DISTINCT TO_CHAR(BR.IssueDate, 'YYYY-MM')) = 12;

```

5. Create View ActiveReceptionist(FName, LName) AS SELECT P.FName, P.LName FROM Person P, Receptionist R WHERE P.PersonID = R.ReceptionistID AND (SELECT COUNT(*) FROM Inquiry I WHERE I.ReceptionistID = R.ReceptionistID AND I.InquiryTime > DATE_SUB(CURDATE(), INTERVAL 1 MONTH) AND I.Status = TRUE) > 5;

```

CREATE OR REPLACE VIEW ActiveReceptionist (FName, LName) AS
SELECT P.FName, P.LName
FROM Person P
JOIN Receptionist R ON P.PersonID = R.ReceptionistID
WHERE (
    SELECT COUNT(*)
    FROM Inquiry I
    WHERE I.ReceptionistID = R.ReceptionistID
        AND I.InquiryTime > ADD_MONTHS(TRUNC(SYSDATE), -1) -- Last 1 month
        AND I.Status = 'Y' -- Changed from TRUE to 'Y' (Oracle uses CHAR for boolean)
) > 5;

```

6. Create View Employee(EmployeeID, Start_Date) AS SELECT EmployeeID, Start_Date FROM CateManager UNION SELECT EmployeeID, Start_Date LibSupervisor UNION SELECT ReceptionistID, Start_Date FROM Receptionist;

```

CREATE OR REPLACE VIEW Employee (EmployeeID, Start_Date) AS
SELECT EmployeeID, Start_Date FROM CateManager
UNION
SELECT EmployeeID, Start_Date FROM LibSupervisor
UNION
SELECT ReceptionistID, Start_Date FROM Receptionist;

```


PART FE

1.

```
SELECT p.PersonID, p.FName, p.MName, p.LName, p.Address, p.Gender, p.DoB
FROM Person p, LibSupervisor l
WHERE l.EmployeeID = p.PersonID AND TRUNC(l.Start_Date) >=
TRUNC(ADD_MONTHS(SYSDATE, -2), 'MONTH');
```

2.

```
SELECT DISTINCT p.FName, p.MName, p.LName, b.BookTitle
FROM Employee e
JOIN Person p ON p.PersonID = e.EmployeeID
JOIN Member m ON m.MemberID = p.PersonID
JOIN Borrow br ON br.MemberID = m.MemberID
JOIN Borrow_Record rec ON rec.RecordID = br.RecordID
JOIN Book b ON b.BookID = br.BookID
WHERE rec.IssueDate >= ADD_MONTHS(SYSDATE, -1);
```

3.

```
WITH gold_counts AS (
    SELECT lc.MemberID,
           COUNT(*) AS borrow_count
    FROM Library_Card lc
         JOIN Borrow br ON br.MemberID = lc.MemberID
         JOIN Borrow_Record rec ON rec.RecordID = br.RecordID
    WHERE lc.MembershipLevel = 'Gold'
    GROUP BY lc.MemberID
),
top5 AS (
    SELECT MemberID, borrow_count,
           ROW_NUMBER() OVER (ORDER BY borrow_count DESC) rn
    FROM gold_counts
)
SELECT AVG(borrow_count) AS avg_books_count
FROM top5
WHERE rn <= 5;
```

4.

```
WITH book_borrow_record AS (
    SELECT b.PublisherID,
           b.BookID,
           b.BookTitle,
```



```

        COUNT(*) AS borrow_count
    FROM    Book    b
    JOIN    Borrow br ON b.BookID = br.BookID
    GROUP BY b.PublisherID, b.BookID, b.BookTitle
),
max_count AS (
    SELECT PublisherID, MAX(borrow_count) AS max_borrow_count
    FROM    book_borrow_record
    GROUP BY PublisherID
)
SELECT pub.PublishName, bbr.BookTitle, bbr.borrow_count
FROM    Publisher    pub
JOIN    book_borrow_record bbr ON pub.PublisherID = bbr.PublisherID
JOIN    max_count    mc ON bbr.PublisherID = mc.PublisherID
        AND bbr.borrow_count = mc.max_borrow_count
ORDER BY pub.PublishName;

```

5.

```

SELECT b.BookTitle
FROM Book b
Where NOT EXIST (
    SELECT 1
    FROM Borrow br
    JOIN BorrowRecord rec ON br.RecordID = rec.RecordID
    WHERE TRUNC(rec.IssueDate) >= TRUNC(ADD_MONTHS(SYSDATE, -5),
'MONTH') AND b.BookID = br.BookID
);

```

6.#define most popular author as the author with most times book being borrowed in total

```

WITH author_borrows AS (
    SELECT c.AuthorID,
        COUNT(*) AS borrow_count
    FROM    Contribute c
    JOIN    Borrow    br ON br.BookID = c.BookID
    GROUP BY c.AuthorID
),
most_pop_author AS (
    SELECT AuthorID
    FROM    author_borrows
    WHERE   borrow_count = (SELECT MAX(borrow_count)
        FROM author_borrows)
)

```

),

```
author_books AS (  
    SELECT DISTINCT c.BookID  
    FROM   Contribute c  
    JOIN   most_pop_author a ON a.AuthorID = c.AuthorID  
)
```

```
member_book_counts AS (  
    SELECT br.MemberID,  
           COUNT(DISTINCT br.BookID) AS cnt  
    FROM   Borrow br  
    JOIN   author_books ab ON ab.BookID = br.BookID  
    GROUP BY br.MemberID
```

),

```
total_author_books AS (  
    SELECT COUNT(DISTINCT BookID) AS total_cnt  
    FROM   author_books  
)
```

```
SELECT p.FName, p.MName, p.LName  
FROM   member_book_counts mbc  
CROSS JOIN total_author_books t  
JOIN   Member m ON m.MemberID = mbc.MemberID  
JOIN   Person p ON p.PersonID = m.MemberID  
WHERE  mbc.cnt = t.total_cnt;
```

7.

```
WITH gold_members AS (  
    SELECT DISTINCT MemberID  
    FROM   Library_Card  
    WHERE  MembershipLevel = 'Gold'
```

),

```
guestCount AS (  
    SELECT gm.MemberID,  
           COUNT(DISTINCT gl.GuestID) AS gCount  
    FROM   gold_members gm  
    LEFT JOIN Guest_Logs gl ON gm.MemberID = gl.MemberID  
    GROUP BY gm.MemberID
```

),

```
maxGuest AS (  
    SELECT MAX(gCount) AS maxgCount  
    FROM   guestCount  
)  
SELECT p.FName, p.MName, p.LName, p.PersonID
```

```

FROM guestCount gc
JOIN maxGuest mg ON gc.gCount = mg.maxgCount
JOIN Member m ON m.MemberID = gc.MemberID
JOIN Person p ON p.PersonID = m.MemberID;

```

8.

```

WITH brcPerYear AS(
    SELECT EXTRACT(YEAR FROM br.IssueDate) as br_year,
           COUNT(*) as br_count
    FROM BorrowRecord br
    GROUP BY EXTRACT(YEAR FROM br.IssueDate)
    ORDER BY br_year
),
maxBorrow AS(
    SELECT MAX(br_count) as max_br_count
    FROM brcPerYear
)
SELECT bcyear.br_year
FROM brcPerYear bcyear
WHERE bcyear.br_count = (
    SELECT max_br_count
    FROM maxBorrow
);

```

9.

```

WITH book_counts AS (
    SELECT br.BookID,
           COUNT(*) AS borrow_count
    FROM Borrow br
    GROUP BY br.BookID
),
max_bc AS (
    SELECT MAX(borrow_count) AS max_count
    FROM book_counts
),
popular_books AS (
    SELECT bc.BookID
    FROM book_counts bc
    JOIN max_bc m ON bc.borrow_count = m.max_count
)
SELECT DISTINCT p.FName, p.MName, p.LName
FROM Borrow br

```

```

JOIN popular_books pb ON pb.BookID = br.BookID
JOIN Member m      ON m.MemberID = br.MemberID
JOIN Person p      ON p.PersonID = m.MemberID;

```

10.

```

SELECT DISTINCT p.FName, p.MName, p.LName
FROM Employee e
JOIN Person p ON p.PersonID = e.EmployeeID
JOIN Library_Card c ON c.MemberID = e.EmployeeID
WHERE c.MembershipLevel = 'Gold'
AND c.IssueDate BETWEEN e.Start_Date AND ADD_MONTHS(e.Start_Date, 1);

```

11.

```

SELECT p.FName, p.MName, p.LName
FROM Person p
JOIN Receptionist r ON p.PersonID = r.ReceptionistID
WHERE (
    Select AVG(i.InquiryRating)
    FROM Inquiry i
    WHERE r.ReceptionistID = i.ReceptionistID
) >= 4.0;

```

12.

```

SELECT p.FName, p.MName, p.LName,
       CASE
         WHEN r.ReceptionistID = p.PersonID THEN 'Receptionist'
         WHEN r.TrainerID = p.PersonID THEN 'Trainer'
       END as Role
FROM Person p
JOIN Receptionist r ON p.PersonID = r.ReceptionistID
WHERE(
    SELECT COUNT(*)
    FROM Inquiry I
    WHERE I.ReceptionistID = R.ReceptionistID
    AND I.InquiryTime >= TRUNC(ADD_MONTHS(SYSDATE, -1), 'MONTH')
    AND I.Status = 'TRUE'
) > 2
AND(
    SELECT COUNT(*)
    FROM Inquiry I

```

```

        WHERE I.ReceptionistID = R.ReceptionistID
        AND I.InquiryTime >= TRUNC(ADD_MONTHS(SYSDATE, -2), 'MONTH')
        AND I.InquiryTime <= TRUNC(ADD_MONTHS(SYSDATE, -1), 'MONTH')
        AND I.Status = 'TRUE'
    ) > 2
    AND(
        SELECT COUNT(*)
        FROM Inquiry I
        WHERE I.ReceptionistID = R.ReceptionistID
        AND I.InquiryTime >= TRUNC(ADD_MONTHS(SYSDATE, -3), 'MONTH')
        AND I.InquiryTime <= TRUNC(ADD_MONTHS(SYSDATE, -2), 'MONTH')
        AND I.Status = 'TRUE'
    ) > 2;

```

13.

```

WITH trainCount AS(
    SELECT t.TrainerID, COUNT(r.ReceptionistID) as trainedCount
    FROM Trainer t
    JOIN Receptionist r on t.TrainerID = r.TrainerID
    GROUP BY t.TrainerID
)
SELECT p.FName, p.MName, p.LName
FROM Person p
JOIN trainCount tc ON p.PersonID = tc.TrainerID
WHERE tc.trainedCount = (
    SELECT MAX(trainedCount)
    FROM trainCount
);

```

14.

```

WITH recent_manages AS (
    SELECT
        m.ManagerID,
        TRUNC(m.Date, 'WW') AS week_start,
        m."Category#" AS CategoryNo
    FROM Manages m
    WHERE m.Date >= TRUNC(SYSDATE) - 28
),

```

```

total_categories AS (
    SELECT COUNT(*) AS cat_cnt
    FROM Category

```

),

```
manager_week_cat AS (  
  SELECT  
    rm.ManagerID,  
    rm.week_start,  
    COUNT(DISTINCT rm.CategoryNo) AS cat_count  
  FROM recent_manages rm  
  GROUP BY rm.ManagerID, rm.week_start  
)
```

```
qualified_weeks AS (  
  SELECT  
    mwc.ManagerID,  
    mwc.week_start  
  FROM manager_week_cat mwc  
  CROSS JOIN total_categories tc  
  WHERE mwc.cat_count = tc.cat_cnt  
)
```

```
qualified_managers AS (  
  SELECT ManagerID  
  FROM qualified_weeks  
  GROUP BY ManagerID  
  HAVING COUNT(DISTINCT week_start) >= 4  
)
```

```
SELECT  
  p.FName,  
  p.MName,  
  p.LName  
FROM qualified_managers qm  
JOIN CateManager cm  
  ON cm.EmployeeID = qm.ManagerID  
JOIN Person p  
  ON p.PersonID = cm.EmployeeID;
```